



Comparing Groups in Traffic Stop Data

The Rhode Island Traffic Stops Dataset, Part Two

Real records from the Stanford Open Policing Project

Nimra Ayaz

Data Scientist & Machine Learning Engineer

Part 2 of a pandas fundamentals walkthrough



RECAP

Picking up where part one left off

- The cleaning notebook left the real file at 86,536 rows and 13 columns
- Every column is complete except search_type, which is only filled in when a search happens
- Five real rows continue to stand in for hands on examples in this notebook
- Every dataset wide statistic below is the real published figure for the complete file

stop_date	driver_gender	driver_race	violation	stop_outcome	district
2005/01/04	M	White	Equipment	Citation	Zone X4
2005/01/23	M	White	Speeding	Citation	Zone K3
2005/02/20	M	White	Other	Arrest Driver	Zone X1
2005/02/24	F	White	Speeding	Citation	Zone X3



SECTION 1

Counting and comparing categories

`value_counts` turns a column of raw values into a table of totals



STEP 1

Count the unique values in a column

```
stops.stop_outcome.value_counts()
```

`value_counts` works best on categorical columns like `stop_outcome`. On the real, complete file it returns these six totals, adding up to all 86,536 stops

Citation	77091
Warning	5136
Arrest Driver	2735
No Action	624
N/D	607
Arrest Passenger	343



STEP 2

Express counts as proportions

77091 / 86536

0.8908546731995932

Dividing a single count by the total gives a proportion.
normalize=True does the same division for every
category at once

Citation 0.890855

Warning 0.059351

Arrest Driver 0.031605

No Action 0.007211

N/D 0.007014

Arrest Passenger 0.003964



STEP 3

Filter the table down to one group

```
stops.driver_race.value_counts()
```

White	61870
Black	12285
Hispanic	9727
Asian	2389
Other	265

```
white = stops[stops.driver_race == 'White']  
white.shape
```

```
(61870, 13)
```

A condition inside square brackets keeps only the rows where it is True



Your turn: count and filter by category

Open a notebook and try each step on your own data



SECTION 2

Filtering by more than one condition

An ampersand for and, a pipe for or, each condition in its own parentheses



STEP 1

Start with a single condition

```
female = stops[stops.driver_gender == 'F']  
female.shape
```

23,774
of 86,536 rows are female
drivers

This filter checks one condition only. The next two slides add a second condition to it



STEP 2

Combine conditions with and

```
female_and_arrested = stops[
    (stops.driver_gender == 'F') &
    (stops.is_arrested == True)]
```

669

female drivers were also arrested



Each condition in its own parentheses

Python needs the grouping to evaluate the comparison first



Ampersand means both must be True

A row only stays if it satisfies every condition joined this way



STEP 3

Combine conditions with or

```
female_or_arrested = stops[  
    (stops.driver_gender == 'F') |  
    (stops.is_arrested == True)]
```

26,183

rows match at least one condition



Includes all females

Every row where driver_gender is F, regardless of the outcome



Includes all arrests

Every arrested driver too, including men who were arrested



REFERENCE

Rules for filtering by multiple conditions

- An ampersand only keeps rows that satisfy both conditions
- A pipe keeps rows that satisfy either condition
- Every condition needs its own set of parentheses
- Conditions can check equality, inequality, and other comparisons
- More than two conditions can be combined the same way



GOOD PRACTICE

Correlation, not causation

90.2%

of White drivers received a citation

92.3%

of Asian drivers received a citation

- This is a real difference in this dataset, worth noting and worth investigating further
- It is not, by itself, evidence of why the difference exists
- Answering a question about cause would need more data and more context than this file provides
- The goal here is only to describe the relationships that appear in the real data



Your turn: filter by more than one condition

Open a notebook and try each step on your own data



SECTION 3

Boolean math and groupby

True and False behave like 1 and 0, which makes group comparisons easy



STEP 1

True and False behave like 1 and 0

```
np.mean([0, 1, 0, 0])  
0.25
```

```
np.mean([False, True,  
         False, False])  
0.25
```

The mean of a Boolean series is the share of values that are True. This works whether the values are literal True and False, or the equivalent 1 and 0



STEP 2

Take the mean of a Boolean column

```
stops.is_arrested.mean()
```

The mean of `is_arrested` is the overall arrest rate. On the real, complete file, this line returns the true arrest rate across every one of the 86,536 stops

3.6%

of all real stops ended in an arrest



STEP 3

Compare arrest rates across zones

```
stops.district.unique()  
<StringArray>  
[ 'Zone X4', 'Zone K3',  
  'Zone X1', 'Zone X3',  
  'Zone K1', 'Zone K2']
```

Rhode Island state police patrol six zones. groupby repeats a calculation for every zone at once, far quicker than filtering one at a time

Zone K1	0.024349
Zone K2	0.030801
Zone K3	0.032311
Zone X1	0.023494
Zone X3	0.034871
Zone X4	0.048038



STEP 4

Group by more than one category

```
stops.groupby(['district',
```

```
'driver_gender']).is_arrested.mean()
```

The order of the list controls how the result is organized, though the underlying numbers stay the same either way

```
district driver_gender
Zone K1  F           0.019169
         M           0.026588
Zone K2  F           0.022196
...      ...           ...
```



GOOD PRACTICE

A caution about small groups



Splitting by more categories shrinks each group
district and driver_gender together create many small subsets



A rate from a handful of stops swings easily
One or two arrests in a tiny group can push a rate to 0 or 1



Check the group size alongside the rate
size() next to mean() shows whether a number rests on real volume



Treat extreme small group rates with care
A dramatic looking number is not automatically a meaningful one



Your turn: compare groups with groupby

Open a notebook and try each step on your own data



SECTION 4

Searching inside a text column

`str.contains` pulls a pattern out of messy text as its own column



STEP 1

Count searches and search types

```
stops.search_conducted  
.value_counts()
```

```
False  83229  
True   3307
```

dropna=False keeps missing values in the count, useful since most stops never involve a search

```
stops.search_type.value_counts(  
    dropna=False)
```

```
NaN          83229  
Incident to Arrest  1290  
Probable Cause    924  
Inventory          219  
Reasonable Suspicion  214  
Protective Frisk   164
```



STEP 2

Look closer at the search types

```
stops.search_type.value_counts()
```

- Leaving dropna at its default drops the missing values automatically
- Some entries list more than one search type, separated by a comma
- Incident to Arrest,Inventory means both reasons applied to that search

Incident to Arrest	1290
Probable Cause	924
Inventory	219
Reasonable Suspicion	214
Protective Frisk	164
Incident to Arrest,Inventory	123
Incident to Arrest,Probable Cause	100



STEP 3

Pull a pattern out with str.contains

```
stops['inventory'] = stops.search_type.str.contains(  
    'Inventory', na=False)
```



True where the text is found

Matches Inventory alone and combined entries like the Incident to Arrest pair



na=False treats a missing value as False

Without it, a missing search_type would raise an error instead

bool

dtype of the new column

441

real stops included on
inventory



STEP 4

Calculate the inventory rate two ways

```
stops.inventory.mean()
```

0.5%

of every stop included an inventory

```
searched = stops[stops.search_conducted ==  
True]  
searched.inventory.mean()
```

13.3%

of searches included an inventory

The second number is the fairer comparison, since an inventory can only happen during a search in the first place

Four ways to compare groups in a real dataset



Count categories

value_counts, with
normalize for proportions



Filter conditions

Ampersand for and, a
pipe for or



Compare with groupby

True and False act as 1
and 0



Search text columns

str.contains pulls out a
pattern

Real records, cleaned and compared: a complete first pass through a public dataset

Source: openpolicing.stanford.edu, Open Data Commons Attribution License